Day 31: Next Greater Element (NGE)

🚀 Next Greater Element (Concept)

👉 Problem:
Har element ke liye right side me pehla bada element find karna hai.

Example:

Input: [4, 5, 2, 10]

Output:
4 → 5
5 → 10
2 → 10
10 → -1

👉 Agar right side me koi greater element nahi mile → -1

🧠 Intuition (Easy Hinglish)

Brute force: Har element ke right me check karo → $O(n^2)$ ❌

Optimal: Stack use karo → $O(n)$ ✅

👉 Trick:

Stack me useful elements store karte hain

Right se left iterate karte hain

⚡ Approach (Step-by-Step)

- ✓ Right se loop chalao
- ✓ Jab tak stack ka top ≤ current element → pop
- ✓ Agar stack empty → answer = -1
  else → answer = stack top
- ✓ Current element ko stack me push karo

🔍 Dry Run

Input: [4, 5, 2, 10]

Start from right:

Element	Stack	Answer
10	[]	-1
2	[10]	10
5	[10]	10
4	[5,10]	5

💻 Java Code (Optimal)

```java
package Day_31;

import java.util.*;

public class NextGreaterElement {

    public static void findNGE(int arr[]) {
        int n = arr.length;
        int result[] = new int[n];

        Stack<Integer> stack = new Stack<>();

        for (int i = n - 1; i >= 0; i--) {

            // Remove smaller elements
            while (!stack.isEmpty() && stack.peek() <= arr[i]) {
                stack.pop();
            }

            // Assign result
            if (stack.isEmpty()) {
                result[i] = -1;
            } else {
                result[i] = stack.peek();
            }

            // Push current element
            stack.push(arr[i]);
        }

        // Print result
        for (int i = 0; i < n; i++) {
            System.out.println(arr[i] + " → " + result[i]);
        }
    }

    public static void main(String[] args) {
        int arr[] = {4, 5, 2, 10};
        findNGE(arr);
    }
}

```

⏱️ Complexity

Time: O(n) ✅

Space: O(n) (stack)